GAME CLIENT PORTFOLIO · TEAM PROJECT

# UnityCrossClassProject

2.5D 좀비 아포칼립스 생존·탐험 게임 — 플레이어 액션 기능 담당

Unity 2022.3 LTS · C# · URP · Cinemachine · Physics Raycast/OverlapSphere · Coroutine · Gizmos · GitHub PR 협업

김동준 · dkdl01028@icloud.com · github.com/kimdongjun22

| | | | |
|---|---|---|---|
| **기간** | 2025.06.18 ~ 2025.07.08 (약 3주) | **인원** | 5인+ 팀 프로젝트 (유니티 사전 합반) |
| **담당** | 플레이어 이동·스탯·전투·건설·은신, 적 탐지 | **저장소** | github.com/JitaeWoo/UnityCrossClassProject |

**11건** PR 전부 병합

**7개** 플레이어 기능 구현

**17개** 직접 작성한 C# 스크립트

**~750줄** C# 코드

## 프로젝트 소개

좀비가 들끓는 세상에서 살아남는 **2.5D 횡스크롤 생존·탐험 게임**입니다. 베이스캠프에서 휴식·요리·제작·수리를 하고, 병원과 학교로 나가 아이템을 모으며 좀비를 피해 숨거나 싸운 뒤 돌아오는 루프로 구성됩니다. 체력·배고픔·수분·정신력 같은 **생존 스탯 관리**가 핵심입니다.

팀은 이니셜 폴더로 작업 공간을 나누고, `이니셜_기능` 형식의 개인 브랜치에서 PR을 올리면 팀장이 리뷰 후 병합하는 규칙을 썼습니다. 저는 **플레이어 액션 기능** 담당으로, 매일 기능 하나씩 브랜치를 따서 PR 11건을 올렸고 모두 병합됐습니다.

## 담당 구현 내용

| 기능 | 클래스 | 구현 방식 |
|---|---|---|
| **3인칭 이동·사다리** | `PlayerController` | 카메라 `right` 벡터 기준 이동(Y 성분 제거 후 정규화), `Ladder` 트리거에 들어가면 중력을 끄고 W/S로 오르내림 |
| **생존 스탯** | `PlayerStatsv2` | 배고픔은 시간에 따라 감소, 피로는 증가. 한계에 닿으면 체력이 지속 감소하고 0이면 사망. `Mathf.Clamp`로 범위 유지 |
| **바리케이드 설치** | `BuildSystem` | `ScreenPointToRay`로 마우스가 가리키는 지면을 찾아 프리팹 설치 |
| **근접 전투** | `PlayerCombat`, `SimpleSword`, `MeleeWeapon`, `WeaponAutoEquip` | 단일 대상은 `Raycast`, 범위 공격은 `OverlapSphere` + `LayerMask`. 시작 시 검을 손 위치에 자동 장착 |
| **무기 쿨타임·내구도** | `WeaponSystem` | 공격 쿨타임, 무기 내구도(0이 되면 파괴), 방어구 내구도 |

| 기능 | 클래스 | 구현 방식 |
| --- | --- | --- |
| **투명화 은신** | `PlayerStealth` | H 키로 머티리얼 알파를 조절하고, 최대 지속시간이 지나면 자동 해제한 뒤 코루틴으로 쿨타임 적용 |
| **캐비닛 숨기 + 적 탐지** | `PlayerHide` , `HideSpot` , `EnemyAI` | 은신처에서 E 키로 숨고 렌더러와 이동을 끔. 정적 `IsHidden` 상태를 적 AI가 읽어 탐지를 건너뜀 |

## 대표 코드

① 은신처 숨기 + 외부에 상태 공개 — PlayerHide.cs

```
public static bool IsHidden { get; private set; }   // 적 AI 등 다른 시스템이 읽는 은신 상태

void EnterHide()
{
    isHiding = true;
    IsHidden = true;
    hideTimer = 0f;
    transform.position = currentSpot.hidePosition.position;  // 은신처 내부로 이동
    SetRenderers(false);                                      // 외형 숨김
    if (controller != null) controller.enabled = false;      // 이동 차단
}

void OnTriggerExit(Collider other)
{
    // 숨으면서 트리거를 벗어나도 은신처 참조를 유지 → 다시 나올 수 있게
    if (other.TryGetComponent(out HideSpot spot) && currentSpot == spot && !isHiding)
        currentSpot = null;
}
```

② 무기 쿨타임·내구도 — WeaponSystem.cs

```
public bool CanAttack()
{
    return Time.time >= lastAttackTime + attackCooldown && durability > 0;
}

public void Attack(Transform attackPoint)
{
    if (!CanAttack()) return;
    lastAttackTime = Time.time;
    durability--;
    Collider[] hitEnemies = Physics.OverlapSphere(attackPoint.position, attackRange, enemyLayer);
    foreach (Collider enemy in hitEnemies)
        enemy.GetComponent<EnemyHealth>()?.TakeDamage(1);
    if (durability <= 0) gameObject.SetActive(false);         // 내구도 소진 시 무기 파괴
}
```

③ 은신 상태와 연동한 적 탐지 — EnemyAI.cs

```
void Update()
{
    if (PlayerHide.IsHidden) return;                    // 숨어 있으면 탐지하지 않음

    Collider[] hits = Physics.OverlapSphere(transform.position, detectionRange, playerLayer);
    foreach (var hit in hits)
        if (hit.CompareTag("Player"))
            Debug.Log("적이 플레이어를 발견함!");
}
```

## 개발 과정 (PR 기록)

| PR | 날짜 | 내용 |
|---|---|---|
| #7 | 06.23 | 3인칭 이동 및 사다리 타기 |
| #11 | 06.24 | 플레이어 생존 스탯 |
| #16 | 06.25 | 바리케이드 설치 |
| #30 · #37 | 06.26 ~ 27 | 근접 무기 공격·적 체력, 공격 시 검 장착 |
| #44 · #60 | 06.27 ~ 30 | 무기 내구도·쿨타임, 전투 스크립트 정리 |
| #72 | 06.30 | 투명화 은신 |
| #77 · #97 | 07.01 ~ 03 | 캐비닛 숨기, 은신처 이탈 버그 수정, 적 AI 탐지 |
| #114 | 07.04 | 숨기 수정, 앉기/일어나기 기초 작업 |

## 문제 해결

- **은신처에서 나올 수 없게 되는 버그**: 숨을 때 플레이어를 은신처 안쪽으로 순간이동시키면 트리거 범위를 벗어나 `OnTriggerExit`가 호출될 수 있습니다. 그러면 은신처 참조가 null이 되어 **다시 나올 수 없게 되는 문제**가 생깁니다. 숨어 있는 동안에는 참조를 유지하도록 `!isHiding` 조건을 추가해 해결했습니다(#97).
- **디버그 시각화**: 공격 범위와 숨는 위치를 `Debug.DrawRay`, `OnDrawGizmosSelected`로 씬 뷰에 그려, 수치를 맞출 때 눈으로 확인할 수 있게 했습니다.
- **반복 개선**: 전투는 `MeleeWeapon` → `SimpleSword` + 자동 장착 → `WeaponSystem`(쿨타임·내구도) 순으로 PR을 거듭하며 기능을 발전시켰습니다.

## 회고 및 개선 계획

- **배운 점**: 기능 하나를 브랜치 하나와 PR 하나로 매일 올리는 흐름을 3주 동안 지켰습니다. 단일 대상 판정(`Raycast`)과 범위 판정(`OverlapSphere`)을 상황에 따라 골라 쓰는 법을 익혔습니다.
- **아쉬운 점 — 최종 빌드 통합**: 제 기능들은 개인 테스트 씬과 초기 파밍 맵에서 동작했지만, 최종 데모 씬에서는 팀원이 만든 공격·은신 구현이 채택되었습니다. 기능을 따로 만들기 전에 **팀의 플레이어 구조(`PlayerStats` partial 클래스 등)에 맞춰 붙이는 방식**으로 작업했어야 했다는 점을 배웠습니다.
- **개선할 점 1 — 결합도**: 정적 `IsHidden` 대신 이벤트(`Action<bool>`)로 은신 상태를 알리도록 바꾸겠습니다.

- **개선할 점 2 — 전투 구조 통합**: 여러 버전으로 남은 전투 클래스를 `IWeapon` 인터페이스 하나로 정리하겠습니다.
- **개선할 점 3 — 협업 규칙**: 팀장 파일에 `NotImplementedException` 스텁을 추가한 적이 있습니다. 앞으로는 **내 폴더 밖 코드가 필요하면 담당자에게 요청**하겠습니다.